

Unit-1

➤ Information based learning

➤ Decision Tress:

Decision trees are a fundamental supervised learning algorithm used in predictive data analytics for both classification and regression tasks. They model decisions and their potential outcomes in a tree-like structure, resembling a flowchart.

- **Structure of a Decision Tree:**

- Root Node:**

- The starting point of the tree, representing the entire dataset and the initial decision or question.

- Internal Nodes:**

- Represent decision rules or tests on specific features of the data, leading to branches based on the outcomes of these tests.

- Branches:**

- Represent the possible outcomes or values of the features tested at internal nodes.

- Leaf Nodes (Terminal Nodes):**

- The end points of the tree, representing the final predicted outcome or class label, with no further splits.

- **Working:**
- Decision trees classify or predict by recursively partitioning the data based on feature values. The algorithm selects the best feature to split the data at each node, aiming to create subsets that are as "pure" as possible (i.e., contain instances predominantly belonging to one class or having similar target values). This process continues until a stopping criterion is met, such as reaching a pure leaf node or a predefined tree depth.
- **Types of Decision Trees:**
- **Classification Trees:**
- Used when the target variable is categorical (e.g., predicting whether an email is spam or not spam).
- **Regression Trees:**
- Used when the target variable is continuous or numerical (e.g., predicting house prices or customer spending).

- **Example:**

Table: By considering An email spam prediction dataset.

ID	SUSPICIOUS WORDS	UNKNOWN SENDER	CONTAINS IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

This dataset contains a set of training instances that can be used to build a model to predict whether emails are spam or ham (genuine). The dataset has three binary descriptive features: SUSPICIOUS WORDS is true if an email contains one or more words that are typically found in spam email (e.g., casino, viagra, bank, or account); UNKNOWN SENDER is true if the email is from an address that is not listed in the contacts of the person who received the email; and CONTAINS IMAGES is true if the email contains one or more images.

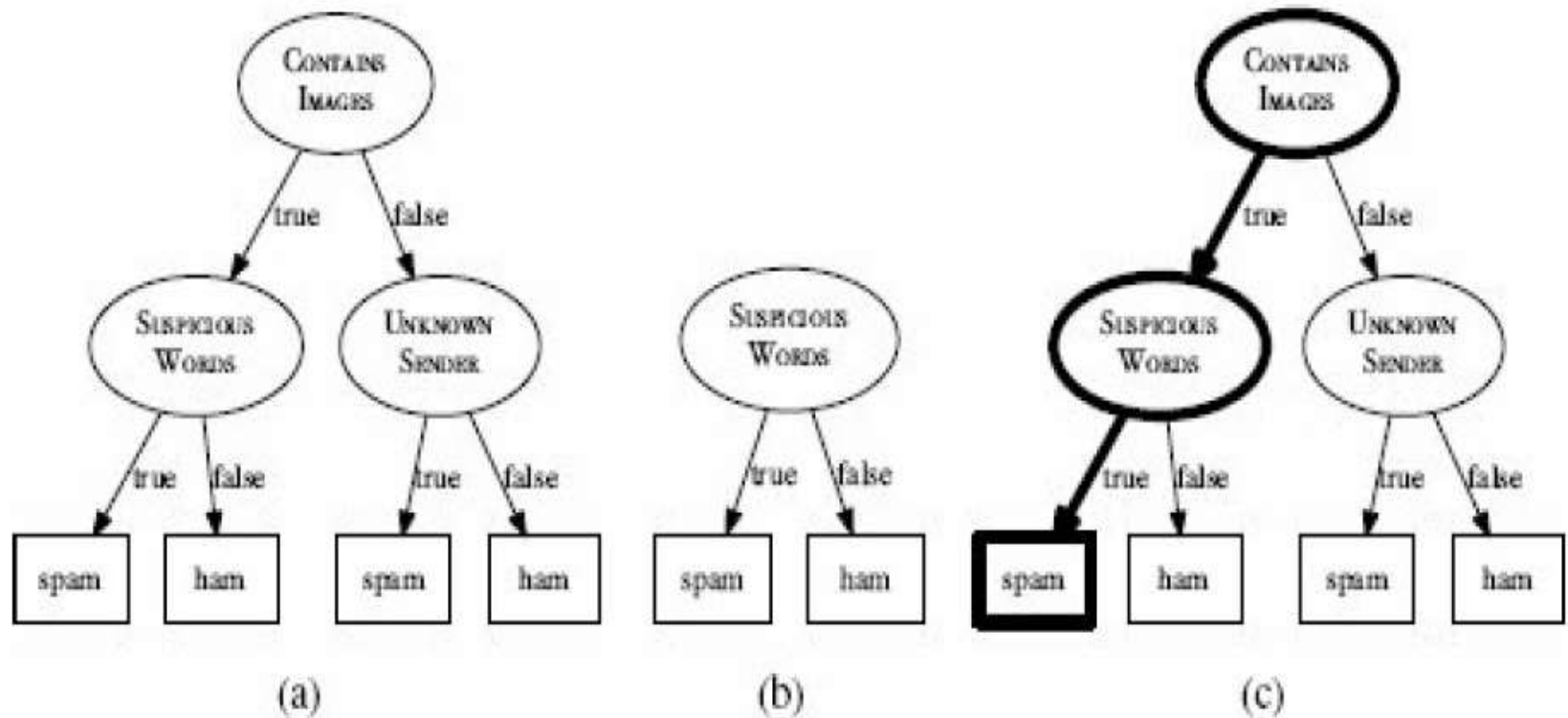


Figure: Two decision trees (a) and (b) that are consistent with the instances in the spam dataset; (c) the path taken through the tree shown in (a) to make a prediction for the query instance *SUSPICIOUS WORDS = true*, *UNKNOWN SENDER = true*, *CONTAINS IMAGES = true*

- Figure above shows two decision trees that are consistent with the spam dataset. Decision trees look very like the game trees that we developed for the *Guess Who game*. As with all tree representations, a decision tree consists of a root node (or starting node), interior nodes, and leaf nodes (or terminating nodes) that are connected by branches. Each non-leaf node (root and interior) in the tree specifies a test to be carried out on a descriptive feature. The number of possible levels that a descriptive feature can take determines the number of downward branches from a non-leaf node. Each of the leaf nodes specifies a predicted level of the target feature. In the diagrams in Figure above, ellipses represent root or interior nodes, and rectangles represent leaf nodes. The labels of the ellipses indicate which descriptive feature is tested at that node. The labels on the each branch indicate one of the possible feature levels that the descriptive feature at the node above can take. The labels on the rectangular leaf nodes indicate the target level that should be predicted when the tests on the interior nodes create a path that terminates at that leaf node.

- The process of using a decision tree to make a prediction for a query instance starts by testing the value of the descriptive feature at the root node of the tree. The result of this test determines which of the root node's children the process should then descend to. These two steps of testing the value of a descriptive feature and descending a level in the tree are then repeated until the process comes to a leaf node at which a prediction can be made.
- To demonstrate how this process works, imagine we were given the query email *SUSPICIOUS WORDS = true, UNKNOWN SENDER = true, CONTAINS IMAGES = true, and asked to predict whether it is spam or ham. Applying the decision tree from Figure (a) to this query, we see that the root node of this tree tests the CONTAINS IMAGES feature. The query instance value for CONTAINS IMAGES is true so the process descends the left branch from the root node, labeled true, to an interior node that tests the SUSPICIOUS WORDS feature. The query instance value for this feature is true, so based on the result of the test at this node, the process descends the left branch, labeled true, to a leaf node labeled spam. As the process has arrived at a leaf node, it terminates, and the target level indicated by the leaf node, spam, is predicted for the query instance. The path through the decision tree for this query instance is shown in figure (C).*

- The decision tree in Figure (b) would have returned the same prediction for the query instance. Indeed, both of the decision trees in Figures (a) and (b) are consistent with the dataset in Table and can generalize sufficiently to make predictions for query instances like the one considered in our example. The fact that there are, at least, two decision trees that can do this raises the question: How do we decide which is the best decision tree to use?
- We can apply almost the same approach that we used in the *Guess Who game* to make this decision. Looking at the decision trees in Figures (a) and (b), we notice that the tree in Figure (a) performs tests on two features in order to make a prediction, while the decision tree in Figure(b) only ever needs to test the value of one feature. The reason for this is that SUSPICIOUS WORDS, the descriptive feature tested at the root node of the tree in Figure (b), perfectly splits the data into a pure group of *spam emails* and a pure group of *ham emails*. We can say that *because of the purity of the splits* that it makes, the SUSPICIOUS WORDS feature provides more information about the value of the target feature for an instance than the CONTAINS IMAGES feature, so a tree that tests this descriptive feature at the root node is preferable.

➤ Shannon's Entropy Model

Shannon's model of entropy is a weighted sum of the logs of the probabilities of each possible outcome when we make a random selection from a set. The weights used in the sum are the probabilities of the outcomes themselves so that outcomes with high probabilities contribute more to the overall entropy of a set than outcomes with low probabilities. Shannon's model of entropy is defined as

Shannon's Entropy Formula

- The formula for Shannon entropy is:

$$H(x) = - \sum P(x_i) * \log_2(P(x_i))$$

- Where:
- **H(x)**: is the Shannon entropy of the random variable x.
- **Σ**: is the summation symbol, meaning you sum the results for all possible events.
- **P(x_i)**: is the probability of the i-th event (x_i) occurring.
- **log₂**: is the logarithm base 2, and the resulting unit of entropy is measured in bits.

The minus sign at the beginning of the equation is simply added to convert the negative numbers returned by the log function to positive ones (as described above).

- **Information Content:**
- Each term $P(x_i) * \log_2(P(x_i))$ represents the information content for a specific event.
- **Logarithm:**
- The logarithm is used because information is additive, meaning the information from two independent events is the sum of their individual information.
- **Probability:**
- If an event has a probability of 1 (or 100%), its contribution to entropy is 0, as there's no uncertainty or new information in a predictable outcome.
- Example: Coin Toss
- Let's analyze a coin toss:
- **1. Fair Coin:**
- A fair coin has two equally likely outcomes: Heads (H) and Tails (T). $P(H) = 0.5$, $P(T) = 0.5$.
- $H(x) = - [(0.5 * \log_2(0.5)) + (0.5 * \log_2(0.5))]$
- $H(x) = - [(0.5 * -1) + (0.5 * -1)] = - [-0.5 - 0.5] = 1 \text{ bit.}$
- **2. Biased Coin:**
- A biased coin where Heads (H) is always the outcome. $P(H) = 1.0$, $P(T) = 0.0$
- $H(x) = - [(1.0 * \log_2(1.0)) + (0.0 * \log_2(0.0))]$
- $H(x) = - [(1.0 * 0) + (0.0 * \text{undefined})] = 0 \text{ bits.}$

- **Interpretation**
- The fair coin toss has an entropy of 1 bit, meaning you need 1 bit of information to represent its outcome.
- The biased coin toss has 0 entropy because there is no uncertainty, and thus no new information

Shannon entropy quantifies the average amount of information, or "surprise," contained in a random variable's outcomes. Higher entropy indicates greater uncertainty and more information is required to describe the event, while lower (or zero) entropy indicates less uncertainty and less information is needed

➤ Information Gain

- Information Gain (IG) is a feature selection metric in information-based learning, such as decision trees, that quantifies how much a feature reduces uncertainty (entropy) about a target variable. It's calculated as the difference between the entropy of a parent node and the weighted average entropy of its child nodes after a split on a feature. Features with higher information gain are considered more informative and are selected for splitting the data, leading to a more homogenous distribution of classes in the resulting nodes.

1. Entropy:

- The fundamental concept is entropy, which measures the impurity or randomness in a dataset. A dataset with 50% of one class and 50% of another has higher entropy than a dataset that is entirely one class.
- 2. Information Gain Calculation:**
- Calculate Parent Entropy:** First, calculate the entropy of the entire dataset before any split.
- Split the Data:** Next, split the dataset into subsets based on the values of a chosen feature.
- Calculate Child Entropy:** Calculate the entropy of each subset (or child node).
- Calculate Weighted Average Entropy:** Compute the weighted average of the child node entropies.
- Determine Information Gain:** Subtract the weighted average child entropy from the parent entropy to get the Information Gain.

- **3. Feature Selection:**

- The feature with the highest Information Gain is chosen as the split point for the node. This process is repeated recursively down the decision tree, selecting the most informative features at each step
- The formula for Information Gain (IG) is:
$$IG(D, A) = H(D) - H(D|A)$$
- $H(D)$ is the entropy of the entire dataset D .
- $H(D|A)$ is the conditional entropy of D given feature A , representing the weighted average entropy after splitting on A

Example:

Imagine a dataset of 10 students and whether they pass (P) or fail (F) based on studying:

Student	Study	Result
1	Yes	P
2	Yes	P
3	Yes	P
4	Yes	P
5	No	F
6	No	F
7	No	F
8	No	P
9	Yes	P
10	No	F

Step 1: Compute entropy of whole dataset

- Pass (P) = 6
- Fail (F) = 4

$$\begin{aligned} H(S) &= -\left(\frac{6}{10} \log_2 \frac{6}{10} + \frac{4}{10} \log_2 \frac{4}{10}\right) \\ &= -(0.6 \cdot -0.737 + 0.4 \cdot -1.322) = 0.971 \text{ bits} \end{aligned}$$

Step 2: Split by "Study"

- If Study = Yes → 5 Pass, 0 Fail → entropy = 0
- If Study = No → 1 Pass, 4 Fail →

$$H(S_{No}) = -\left(\frac{1}{5} \log_2 \frac{1}{5} + \frac{4}{5} \log_2 \frac{4}{5}\right) = 0.722$$

Step 3: Weighted average entropy after split

$$H_{split} = \frac{5}{10}(0) + \frac{5}{10}(0.722) = 0.361$$

Step 4: Compute Information Gain

$$IG(S, Study) = H(S) - H_{split} = 0.971 - 0.361 = 0.610 \text{ bits}$$

Interpretation

IG = 0.610 bits → Knowing the attribute "**Study**" reduces uncertainty by more than half.

A decision tree would select **Study** as the root node for splitting.

➤ ID3 Algorithm

One of the best known decision tree induction algorithms is the Iterative Dichotomizer 3 (ID3) algorithm.⁸ This algorithm attempts to create the shallowest decision tree that is consistent with the data given. The ID3 algorithm builds the tree in a recursive, depth-first manner, beginning at the root node and working down to the leaf nodes. The algorithm begins by choosing the best descriptive feature to test (i.e., the best question to ask first). This choice is made by computing the information gain of the descriptive features in the training dataset. A root node is then added to the tree and labeled with the selected test feature. The training dataset is then partitioned using the test. There is one partition created for each possible test result, which contains the training instances that returned that result. For each partition a branch is grown from the node. The process is then repeated for each branch using the relevant partition of the training set in place of the full training set and with the selected test feature excluded from further testing. This process is repeated until all the instances in a partition have the same target level, at which point a leaf node is created and labeled with that level.

The design of the ID3 algorithm is based on the assumption that a correct decision tree for a domain will classify instances from that domain in the same proportion as the target level occurs in the domain. So, given a dataset D representing a domain with two target levels $C1$ and $C2$, *an arbitrary instance from the domain should be classified as being associated with target level $C1$ with the probability $\frac{|C1|}{|C1|+|C2|}$ and to target level $C2$ with*

the probability $\frac{|C2|}{|C1|+|C2|}$, where $|C1|$ and $|C2|$ refer to the number of instances in associated with $C1$ and $C2$ respectively. To ensure that the resulting decision tree classifies in the correct proportions, the decision tree is constructed by repeatedly partitioning the training dataset, until every instance in a partition maps to the same target level.

Algorithms lists a pseudocode description of the ID3 algorithm. Although the algorithm looks quite complex, it essentially does one of two things each time it is invoked: it either stops growing the current path in the tree by adding a leaf node to the tree, Lines 1–6, or it extends the current path by adding an interior node to the tree and growing the branches of this node by repeatedly rerunning the algorithm, Lines 7–13.

Algorithm	Pseudocode description of the ID3 algorithm.
-----------	--

Require: set of descriptive features d

Require: set of training instances $\mathcal{D}$

- 1: if all the instances in $\mathcal{D}$ have the same target level C then
 - 2: return a decision tree consisting of a leaf node with label C
 - 3: else if d is empty then
 - 4: return a decision tree consisting of a leaf node with the label of the majority target
 - 5: else if $\mathcal{D}$ is empty then
 - 6: return a decision tree consisting of a leaf node with the label of the majority target
 level of the dataset of the immediate parent node
 - 7: else
 - 8: $d [best] \leftarrow \arg \max_{d \in d} IG(d, \mathcal{D})$
 - 9: make a new node, $Node_{d[best]}$, and label it with $d [best]$
 - 10: partition $\mathcal{D}$ using $d [best]$
 - 11: remove $d [best]$ from d
 - 12: for each partition $\mathcal{D}_i$ of $\mathcal{D}$ do
 - 13: grow a branch from $Node_{d[best]}$ to the decision tree created by rerunning $ID3$ with $\mathcal{D}$
 $= \mathcal{D}_i$
-

Lines 1–6 of Algorithm control when a new leaf node is created in the tree. We have already mentioned that the ID3 algorithm constructs the decision tree by recursively partitioning the dataset. An important decision to be made when designing any recursive process is what the base cases that stop the recursion will be. In the ID3 algorithm the base cases are the situations where we stop splitting the dataset and construct a leaf node with an associated target level. There are two important things to remember when designing these base cases. First, the dataset of training instances considered at each of the interior nodes in the tree is not the complete dataset; rather, it is the subset of instances considered at its parent node that had the relevant feature value for the branch from the parent to the current node. Second, once a feature has been tested, it is not considered for selection again along that path in the tree. A feature will only be tested once on any path in the tree, but it may occur several times in the tree on different paths.

Based on these constraints, the algorithm defines three situations where the recursion stops and a leaf node is constructed:

1. All the instances in the dataset have the same target feature level. In this situation, the algorithm returns a single leaf node tree with that target level as its label (Algorithm Lines 1–2).
2. The set of features left to test is empty. This means that we have already tested every feature on the path between the root node and the current node. We have no more features we can use to distinguish between the instances, so we return a single leaf node tree with the majority target level of the dataset as its target level (Algorithm Lines 3–4).
3. The dataset is empty. This can occur when, for a particular partition of the dataset, there are no instances that have a particular feature value. In this case we return a single leaf node tree with the majority target level of the dataset at the parent node that made the recursive call (Algorithm Lines 5–6).

If none of these cases hold, the algorithm continues to recursively create interior nodes, Lines 7–13 of Algorithm . The first step in creating an interior node is to decide which descriptive feature should be tested at this node (Line 8 of Algorithm).

When we first mentioned the ID3 algorithm, we stated that it tries to create the shallowest decision tree that is consistent with the data given. The feature of the ID3 algorithm that biases it toward shallow trees is the mechanism that it uses to determine which descriptive feature is the *most informative one to test at a new node*. *The ID3 algorithm uses the* information gain metric to choose the best feature to test at each node in the tree.

Note: Consider any example and construct a decision tree using ID3 algorithm

➤ **Extensions and Variations**

The ID3 decision tree induction algorithm described in the previous section provides the basic approach to decision tree induction: a top-down, recursive, depth-first partitioning of the dataset beginning at the root node and finishing at the leaf nodes. Although this algorithm works quite well as presented, it assumes categorical features and clean data. It is relatively easy, however, to extend the ID3 algorithm to handle continuous descriptive features and continuous target features. A range of techniques can also be used to make a decision tree more robust to noise in the data.

➤ Alternative Feature Selection and Impurity Metrics

The information gain measure described uses entropy to judge the impurity of the partitions that result from splitting a dataset using a particular feature. Entropy-based information gain, however, does have some drawbacks. In particular, it preferences features with many levels because these features will split the data into many small subsets, which will tend to be pure, irrespective of any correlation between the descriptive feature and the target feature. One way of addressing this issue is to use information gain ratio instead of entropy. The information gain ratio is computed by dividing the information gain of a feature by the amount of information used to determine the value of the feature

$$GR(d, \mathcal{D}) = \frac{IG(d, \mathcal{D})}{-\sum_{l \in \text{levels}(d)} (P(d=l) \times \log_2(P(d=l)))}$$

- where $IG(d,)$ is the information gain of the feature d for the dataset, and the divisor is the entropy of the dataset with respect to the feature d (note that $\text{levels}(d)$ is the set of levels that the feature d can take). This divisor biases information gain ratio away from features that take on a large number of values and as such counteracts the bias in information gain toward these features.

- To illustrate how information gain ratio is computed, we will compute the information gain ratio for the descriptive features STREAM, SLOPE, and ELEVATION in the vegetation classification dataset . We already know the information gain for these features Table below

Split by Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	<i>true</i>	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	1.5000	1.2507	0.3060
	<i>false</i>	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.9183		
SLOPE	<i>flat</i>	$\mathcal{D}_3$	$\mathbf{d}_5$	0.0	0.9793	0.5774
	<i>moderate</i>	$\mathcal{D}_4$	$\mathbf{d}_2$	0.0		
	<i>steep</i>	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	1.3710		
ELEVATION	<i>low</i>	$\mathcal{D}_6$	$\mathbf{d}_2$	0.0	0.6793	0.8774
	<i>medium</i>	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	1.0		
	<i>high</i>	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.9183		
	<i>highest</i>	$\mathcal{D}_9$	$\mathbf{d}_6$	0.0		

$$IG(\text{STREAM}, \mathcal{D}) = 0.3060$$

$$IG(\text{SLOPE}, \mathcal{D}) = 0.5774$$

$$IG(\text{ELEVATION}, \mathcal{D}) = 0.8774$$

To convert these information gain scores into information gain ratios, we need to compute the entropy of each feature and then divide the information gain scores by the respective entropy values. The entropy calculations for these descriptive features are

$$\begin{aligned}
H(\text{STREAM}, \mathcal{D}) &= - \sum_{l \in \{\text{true}, \text{false}\}} P(\text{STREAM} = l) \times \log_2(P(\text{STREAM} = l)) \\
&= - \left(\left(\frac{4}{7} \times \log_2 \left(\frac{4}{7} \right) \right) + \left(\frac{3}{7} \times \log_2 \left(\frac{3}{7} \right) \right) \right) \\
&= 0.9852 \text{ bits}
\end{aligned}$$

$$\begin{aligned}
H(\text{SLOPE}, \mathcal{D}) &= - \sum_{l \in \left\{ \begin{smallmatrix} \textit{flat}, \\ \textit{moderate}, \\ \textit{steep} \end{smallmatrix} \right\}} P(\text{SLOPE} = l) \times \log_2(P(\text{SLOPE} = l)) \\
&= - \left(\left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) + \left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) \right. \\
&\quad \left. + \left(\frac{5}{7} \times \log_2 \left(\frac{5}{7} \right) \right) \right) \\
&= 1.1488 \text{ bits}
\end{aligned}$$

$$\begin{aligned}
H(\text{ELEVATION}, \mathcal{D}) &= - \sum_{l \in \left\{ \begin{smallmatrix} \textit{low}, \\ \textit{medium}, \\ \textit{high}, \\ \textit{highest} \end{smallmatrix} \right\}} P(\text{ELEVATION} = l) \times \log_2(P(\text{ELEVATION} = l)) \\
&= - \left(\left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) + \left(\frac{2}{7} \times \log_2 \left(\frac{2}{7} \right) \right) \right. \\
&\quad \left. + \left(\frac{3}{7} \times \log_2 \left(\frac{3}{7} \right) \right) + \left(\frac{1}{7} \times \log_2 \left(\frac{1}{7} \right) \right) \right) \\
&= 1.8424 \text{ bits}
\end{aligned}$$

Using these results, we can now compute the information gain ratio for each descriptive feature by dividing the feature's information gain by the entropy for that feature:

$$GR(\text{STREAM}, \mathcal{D}) = \frac{0.3060}{0.9852} = 0.3106$$
$$GR(\text{SLOPE}, \mathcal{D}) = \frac{0.5774}{1.1488} = 0.5026$$
$$GR(\text{ELEVATION}, \mathcal{D}) = \frac{0.8774}{1.8424} = 0.4762$$

From these calculations we can see that SLOPE has the highest information gain ratio score, even though ELEVATION has the highest information gain. The implication of this is that if we build a decision tree for the dataset

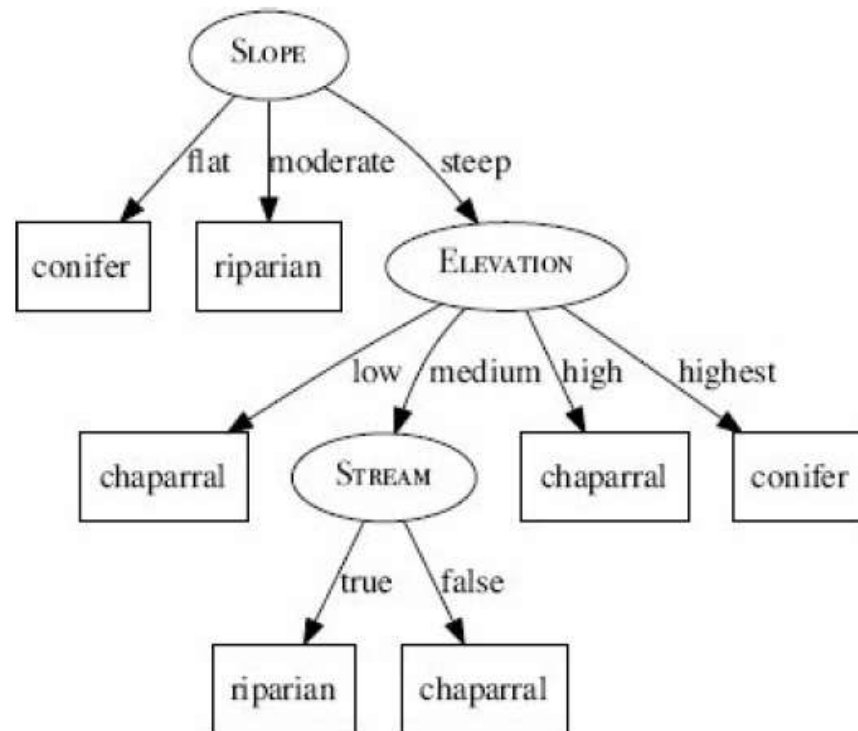


Figure: The vegetation classification decision tree generated using information gain ratio.

Another commonly used measure of impurity is the Gini index:

$$Gini(t, \mathcal{D}) = 1 - \sum_{l \in \text{levels}(t)} P(t = l)^2$$

Where $\mathcal{D}$ is a dataset with a target feature t ; $\text{levels}(t)$ is the set of levels in the domain of the target feature; and $P(t = l)$ is the probability of an instance of $\mathcal{D}$ having the target level l . The Gini index can be understood as calculating how often the target levels of instances in a dataset would be misclassified if predictions were made based only on the distribution of the target levels in the dataset. For example, if there were two target levels with equal likelihood in a dataset, then the expected rate of misclassification would be 0.5, and if there were four target levels with equal likelihood, then the expected rate of misclassification would be 0.75. The Gini index is 0 when all the instances in the dataset have the same target level and $1 - \frac{1}{k}$ when there are k possible target levels with equal likelihood. Indeed, a nice feature of the Gini index is that Gini index scores are always between 0 and 1, and in some contexts this may make it easier to compare Gini indexes across features.

➤ Handling Continuous Descriptive Features

The easiest way to handle a continuous descriptive feature in a decision tree is to define a threshold within the range of values that the continuous feature can take and use this threshold to partition the instances based on whether their values for the feature are above or below the threshold. The only challenge is to determine the best threshold to use.

Ideally, we should use the threshold that results in the highest information gain when the feature is used to split the dataset. The problem, however, is that with a continuous feature, there is an infinite number of thresholds to choose from.

There is, though, a simple way to find the optimal threshold, which avoids testing an infinite number of possible thresholds. First, the instances in the dataset are sorted according to the values of the continuous feature. The adjacent instances in the ordering that have different target feature levels are then selected as possible threshold points. It can be shown that the optimal threshold value must lie at one of the boundaries between adjacent instances with different target levels. The optimal threshold is found by computing the information gain for each of the target level transition boundaries and selecting the boundary with the highest information gain as the threshold. Once a threshold has been set, the continuous feature can compete with the other categorical features for selection as the splitting feature at any node. To illustrate how this is done, we will use a modified version of the vegetation classification dataset in which the ELEVATION feature now contains actual elevations in feet. This dataset is listed in Table below

Dataset for predicting the vegetation in an area with a continuous ELEVATION feature (measured in feet).

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	3,900	chapparal
2	true	moderate	300	riparian
3	true	steep	1,500	riparian
4	false	steep	1,200	chapparal
5	false	flat	4,450	conifer
6	true	steep	5,000	conifer
7	true	steep	3,000	chapparal

To select the best feature to use at the root of the tree, we need to calculate the information gain for each feature. We know from our earlier calculations that the entropy for this dataset is 1.5567 bits (see Equation (4.5)) and that the information gain for the categorical features are $IG(STREAM,) = 0.3060$ and $IG(SLOPE,) = 0.5774$ (see Table 4.4). This leaves us with the tasks of calculating the best threshold on which to split the ELEVATION feature, and calculating the information gain when we partition the dataset with ELEVATION using this optimal threshold. Our first task, is to sort the dataset based on the ELEVATION feature. This is shown in Table 4.9

Dataset for predicting the vegetation in an area sorted by the continuous ELEVATION feature.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
2	true	moderate	300	riparian
4	false	steep	1,200	chapparal
3	true	steep	1,500	riparian
7	true	steep	3,000	chapparal
1	false	steep	3,900	chapparal
5	false	flat	4,450	conifer
6	true	steep	5,000	conifer

- Once the instances have been sorted, we look for adjacent pairs that have different target levels. In Table above we can see that four pairs of adjacent instances have a transition between the target levels, instances d2 and d4, d4 and d3, d3 and d7, and d1 and d5.

- The boundary value between each of these pairs is simply the average of their ELEVATION values:

- the boundary between d_2 and d_4 is $\frac{300 + 1,200}{2} = 750$
- the boundary between d_4 and d_3 is $\frac{1,200 + 1,500}{2} = 1,350$
- the boundary between d_3 and d_7 is $\frac{1,500 + 3,000}{2} = 2,250$
- the boundary between d_1 and d_5 is $\frac{3,900 + 4,450}{2} = 4,175$

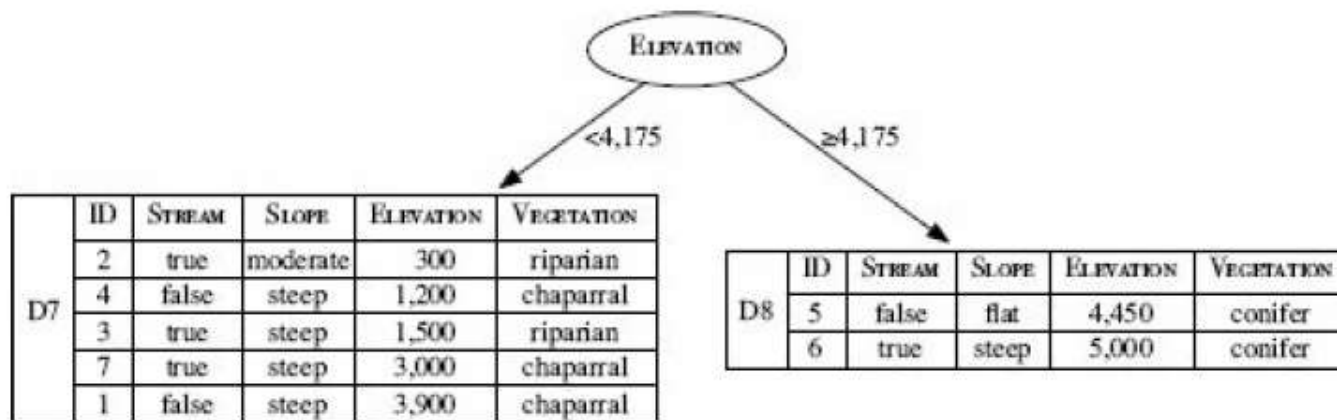


Figure 2

The vegetation classification decision tree after the dataset has been split using $ELEVATION \geq 4,175$.

Table

Partition sets (Part.), entropy, remainder (Rem.), and information gain (Info. Gain) for the candidate ELEVATION thresholds: ≥ 750 , $\geq 1,350$, $\geq 2,250$ and $\geq 4,175$.

Split by Threshold	Part.	Instances	Partition Entropy	Rem.	Info. Gain
≥ 750	$\mathcal{D}_1$	d_2	0.0	1.2507	0.3060
	$\mathcal{D}_2$	$d_4, d_3, d_7, d_1, d_5, d_6$	1.4591		
$\geq 1,350$	$\mathcal{D}_3$	d_2, d_4	1.0	1.3728	0.1839
	$\mathcal{D}_4$	d_3, d_7, d_1, d_5, d_6	1.5219		
$\geq 2,250$	$\mathcal{D}_5$	d_2, d_4, d_3	0.9183	0.9650	0.5917
$\geq 2,250$	$\mathcal{D}_6$	d_7, d_1, d_5, d_6	1.0	0.9650	0.5917
$\geq 4,175$	$\mathcal{D}_7$	d_2, d_4, d_3, d_7, d_1	0.9710	0.6935	0.8631
	$\mathcal{D}_8$	d_5, d_6	0.0		

This results in four candidate thresholds: ≥ 750 , $\geq 1,350$, $\geq 2,250$, and $\geq 4,175$. Table above shows the computation of information gain for a split using each of these thresholds. The threshold $\geq 4,175$ has the highest information gain of any of the candidate thresholds (0.8631 bits), and this information gain is also higher than the information gain for either of the other two descriptive features. So, we should use $\text{ELEVATION} \geq 4,175$ as the test at the root node of the tree, as shown in Figure above

➤ Predicting Continuous Targets

It is relatively easy to adapt the *k nearest neighbor approach to handle continuous target features*. To do this we simply change the approach to return a prediction of the average target value of the nearest neighbors, rather than the majority target level. The prediction for a continuous target feature by a *k nearest neighbor model is therefore*

$$\mathbb{M}_k(\mathbf{q}) = \frac{1}{k} \sum_{i=1}^k t_i \quad \text{----- equ(1)}$$

where $M_k(q)$ is the prediction returned by the model using parameter value k for the query q , i iterates over the k nearest neighbors to q in the dataset, and t_i is the value of the target feature for instance i .

Let's look at an example. Imagine that we are dealers in rare whiskey, and we would like some assistance in setting the reserve price for bottles of whiskey that we are selling at auction. We can use a *k nearest neighbor model to predict the likely sale price of a bottle of whiskey* based on the prices achieved by similar bottles at previous auctions. Table 5.8 lists a dataset of whiskeys described by the RATING they were given in popular whiskey enthusiasts magazine and their AGE (in years). The PRICE achieved at auction by the each bottle is also included.

Table 5.8

A dataset of whiskeys listing the age (in years), the rating (between 1 and 5, with 5 being the best), and the bottle price of each whiskey.

ID	AGE	RATING	PRICE
1	0	2	30.00
2	12	3.5	40.00
3	10	4	55.00
4	21	4.5	550.00
5	12	3	35.00
6	15	3.5	45.00
7	16	4	70.00
8	18	3	85.00
9	18	3.5	78.00
10	16	3	75.00
11	19	5	500.00
12	6	4.5	200.00
13	8	3.5	65.00
14	22	4	120.00
15	6	2	12.00
16	8	4.5	250.00
17	10	2	18.00
18	30	4.5	450.00
19	1	1	10.00
20	4	3	30.00

One thing that is immediately apparent in Table above is that the AGE and RATING features have different ranges. We should normalize these features before we build a model. Table 5.9 lists the whiskey dataset after the descriptive features have been normalized, using range normalization to the range [0, 1].

Let's now make a prediction using this model for a 2-year-old bottle of whiskey that received a magazine rating of 5. Having normalized the dataset, we first need to normalize the descriptive feature values of this query instance using the same normalization process.

This results in a query with AGE = 0.0667 and RATING = 1.00. For this example we set $k=3$. Figure 5.13 shows the neighborhood that this defines around the query instance. The three closest neighbors to the query are instances d_{12} , d_{16} and d_3 . Consequently, the model will return a price prediction that is the average price of these three neighbors:

$$\mathbb{M}_3(\langle 0.0667, 1.00 \rangle) = \frac{200.00 + 250.00 + 55.00}{3} = 168.33$$

Table 5.9

The whiskey dataset after the descriptive features have been normalized.

ID	AGE	RATING	PRICE
1	0.0000	0.25	30.00
2	0.4000	0.63	40.00
3	0.3333	0.75	55.00
4	0.7000	0.88	550.00
5	0.4000	0.50	35.00
6	0.5000	0.63	45.00
7	0.5333	0.75	70.00
8	0.6000	0.50	85.00
9	0.6000	0.63	78.00
10	0.5333	0.50	75.00
11	0.6333	1.00	500.00
12	0.2000	0.88	200.00
13	0.2667	0.63	65.00
14	0.7333	0.75	120.00
15	0.2000	0.25	12.00
16	0.2667	0.88	250.00
17	0.3333	0.25	18.00
18	1.0000	0.88	450.00
19	0.0333	0.00	10.00
20	0.1333	0.50	30.00

We can also use a weighted *k nearest neighbor model to make predictions for* continuous targets that take into account the distance from the query instance to the neighbors . To do this, the model prediction equation in Equ(1) is changed to

$$\mathbb{M}_k(\mathbf{q}) = \frac{\sum_{i=1}^k \frac{1}{\text{dist}(\mathbf{q}, \mathbf{d}_i)^2} \times t_i}{\sum_{i=1}^k \frac{1}{\text{dist}(\mathbf{q}, \mathbf{d}_i)^2}} \quad \text{----- equ(2)}$$

where *dist(q, di)* is the distance between the query instance and its *ith* nearest neighbor. This is a weighted average of the target values of the *k* nearest neighbors, as opposed to the simple average in Equ (1).

Table 5.10 shows the calculation of the numerator and denominator of Equ (2) for our whiskey bottle example, using the normalized dataset with *k* set to 20 (the full size of the dataset). The final prediction for the price of the bottle of whiskey we plan to sell is

$$\frac{16,249.85}{99.2604} = 163.71$$

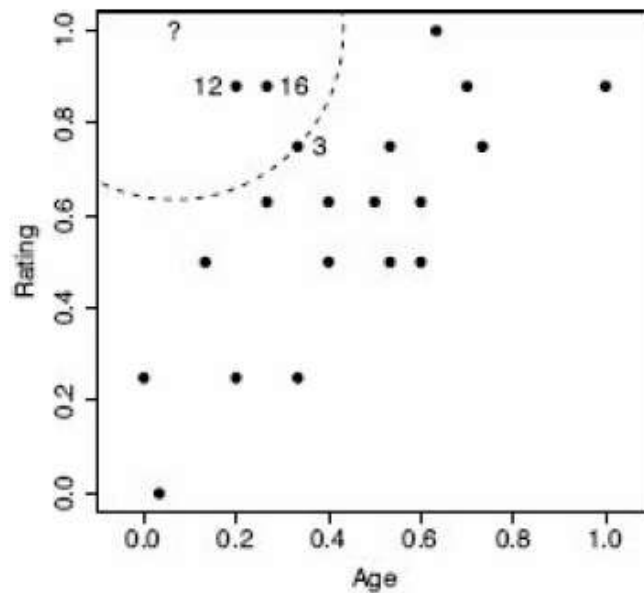


Figure 5.13: The AGE and RATING feature space for the whiskey dataset. The location of the query instance is indicated by the ? symbol. The circle plotted with a dashed line demarcates the border of the neighborhood around the query when $k = 3$. *The three nearest neighbors to the query are labeled with their ID values.*

The predictions using the $k = 3$ nearest neighbor model and the weighted k nearest neighbor model with k set to the size of the dataset are quite similar: 168.33 and 163.71. So, which model is making the better prediction? In this instance, to find out which model is best, we would really need to put the bottle of whiskey up for auction and see which model predicted the closest price. In situations where we have a larger dataset, however, we could perform evaluation experiments to see which value of k leads to the best performing model. In general, standard k nearest neighbor models and weighted k nearest neighbor models will produce very similar results when a feature space is well populated. For datasets that only sparsely populate the feature space, however, weighted k nearest neighbor models usually make more accurate predictions as they take into account the fact that some of the nearest neighbors can actually be quite far away.

Table 5.10

The calculations for the weighted k nearest neighbor prediction.

ID	PRICE	Distance	Weight	PRICE $\times$ Weight
1	30.00	0.7530	1.7638	52.92
2	40.00	0.5017	3.9724	158.90
3	55.00	0.3655	7.4844	411.64
4	550.00	0.6456	2.3996	1319.78
5	35.00	0.6009	2.7692	96.92
6	45.00	0.5731	3.0450	137.03
7	70.00	0.5294	3.5679	249.75
8	85.00	0.7311	1.8711	159.04
9	78.00	0.6520	2.3526	183.50
10	75.00	0.6839	2.1378	160.33
11	500.00	0.5667	3.1142	1557.09
12	200.00	0.1828	29.9376	5987.53
13	65.00	0.4250	5.5363	359.86
14	120.00	0.7120	1.9726	236.71
15	12.00	0.7618	1.7233	20.68
16	250.00	0.2358	17.9775	4494.38
17	18.00	0.7960	1.5783	28.41
18	450.00	0.9417	1.1277	507.48
19	10.00	1.0006	0.9989	9.99
20	30.00	0.5044	3.9301	117.90
		Totals:	99.2604	16,249.85



Tree Pruning

A predictive model overfits the training set when at least some of the predictions it returns are based on spurious patterns present in the training data used to induce the model.

Overfitting happens for a number of reasons, including sampling variance¹⁸ and noise in the training set.¹⁹ The problem of overfitting can affect any machine learning algorithm; however, the fact that decision tree induction algorithms work by recursively splitting the training data means that they have a natural tendency to segregate noisy instances and to create leaf nodes around these instances. Consequently, decision trees overfit by splitting the data on irrelevant features that only appear relevant due to noise or sampling variance in the training data. The likelihood of overfitting occurring increases as a tree gets deeper because the resulting predictions are based on smaller and smaller subsets as the dataset is partitioned after each feature test in the path.

Tree pruning identifies and removes subtrees within a decision tree that are likely to be due to noise and sample variance in the training set used to induce it. In cases where a subtree is deemed to be overfitting, pruning the subtree means replacing the subtree with a leaf node that makes a prediction based on the majority target feature level (or average target feature value) of the dataset created by merging the instances from all the leaf nodes in the subtree. Obviously, pruning will result in decision trees being created that are not consistent with the training set used to build them. In general, however, we are more interested in creating prediction models that generalize well to new data rather than that are strictly consistent with training data, so it is common to sacrifice consistency for generalization capacity.

The simplest way to prune a decision tree is to introduce early stopping criteria (similar to the one discussed in the previous section) into the tree induction algorithm. This is often known as pre-pruning. There are a range of simple pre-pruning strategies. For example, we can stop creating subtrees when the number of instances in a partition falls below a threshold, when the information gain (or whatever other feature selection metric is being used) measured at a node is not deemed to be sufficient to make partitioning the data worthwhile,²⁰ or when the depth of the tree goes beyond a predefined limit. More advanced approaches to pre-pruning use statistical significance tests to determine the importance of subtrees, for example, χ^2 pruning (pronounced *chi-squared*)

Pre-pruning approaches are computationally efficient and can work well for small datasets. By stopping the partitioning of the data early, however, induction algorithms that use pre-pruning can fail to create the most effective trees because they miss interactions between features that emerge within subtrees that are not obvious when the parent nodes are being considered. Pre-pruning can mean that these useful subtrees are never created.

Post-pruning is an alternative approach to tree pruning in which the tree induction algorithm is allowed to grow a tree to completion, and then each branch on the tree is examined in turn. Branches that are deemed likely to be due to overfitting are pruned. Post-pruning relies on a criteria that can distinguish between subtrees that model relevant aspects of the data and subtrees that model irrelevant random patterns in the data. There are a range of different criterion that can be used from a very simple threshold on the number of instances at a node in the tree, to statistical significance tests like χ^2 .

We recommend the use of criteria that compare the error rate in the predictions made by a decision tree when a given subtree is included and when it is pruned. To measure error rate, we set aside some of the training data as a validation dataset²² that is not used during tree induction. We can measure the performance of a decision tree by presenting the instances in the validation to the decision tree and comparing the predictions made for these instances with the actual target feature values in the dataset. The error rate measures the number of predictions made by the tree that are incorrect. A subtree is pruned if the error rate on the validation set of the decision tree with the subtree removed is no greater than the error rate of the decision tree when the subtree is included. Because the instances in the validation set are not used during training, the error rate on the validation set provides a good estimate of the generalization capability of a decision tree.

Reduced error pruning (Quinlan, 1987) is a popular version of post-pruning based on error rates. In reduced error pruning, a decision tree is built to completion and then the tree is searched in an iterative, bottom-up, left-to-right manner for subtrees that can be pruned. The error rate resulting from predictions for the instances in the validation dataset made at the root node of each subtree is compared to the error rate resulting from predictions made at the leaves of the subtree. If the error rate at the subtree root node is less than or equal to the combined error rate at the leaves, the subtree is pruned.

To show how reduced error pruning works, we will consider the task of predicting whether a post-operative patient should be sent to an intensive care unit (ICU) or to a general ward for recovery.²³ Hypothermia is a major concern for post-operative patients, so many of the descriptive features relevant to this domain relate to a patient's body temperature. In our example CORE-TEMP describes the core temperature of the patient (which can be *low* or *high*) and STABLE-TEMP describes whether the patient's current temperature is stable (*true* or *false*). We also include the GENDER of the patient (*male* or *female*). The target feature in this domain, DECISION, records the decision of whether the patient is sent to the *icu* or to a general ward (*gen*) for recovery. Figure below illustrates a decision tree that has been trained for this post-operative patient routing task. The target level in square brackets at each interior node in the tree shows the majority target level for the data partition at that node.

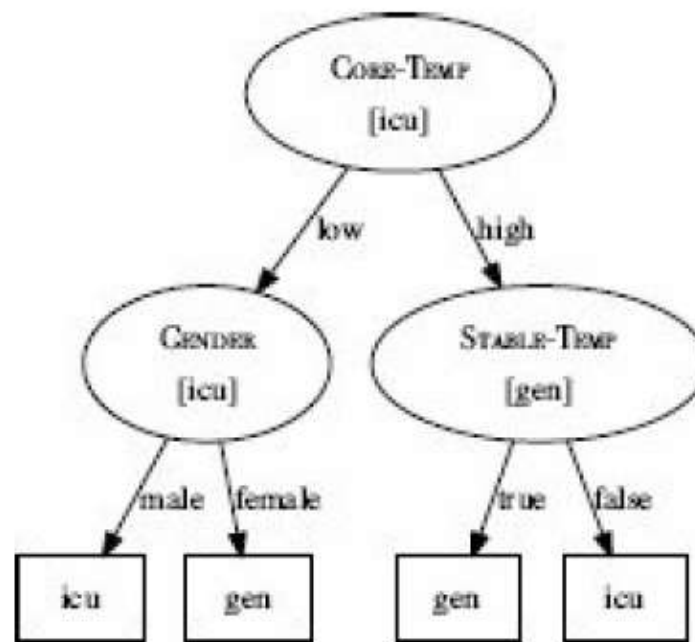


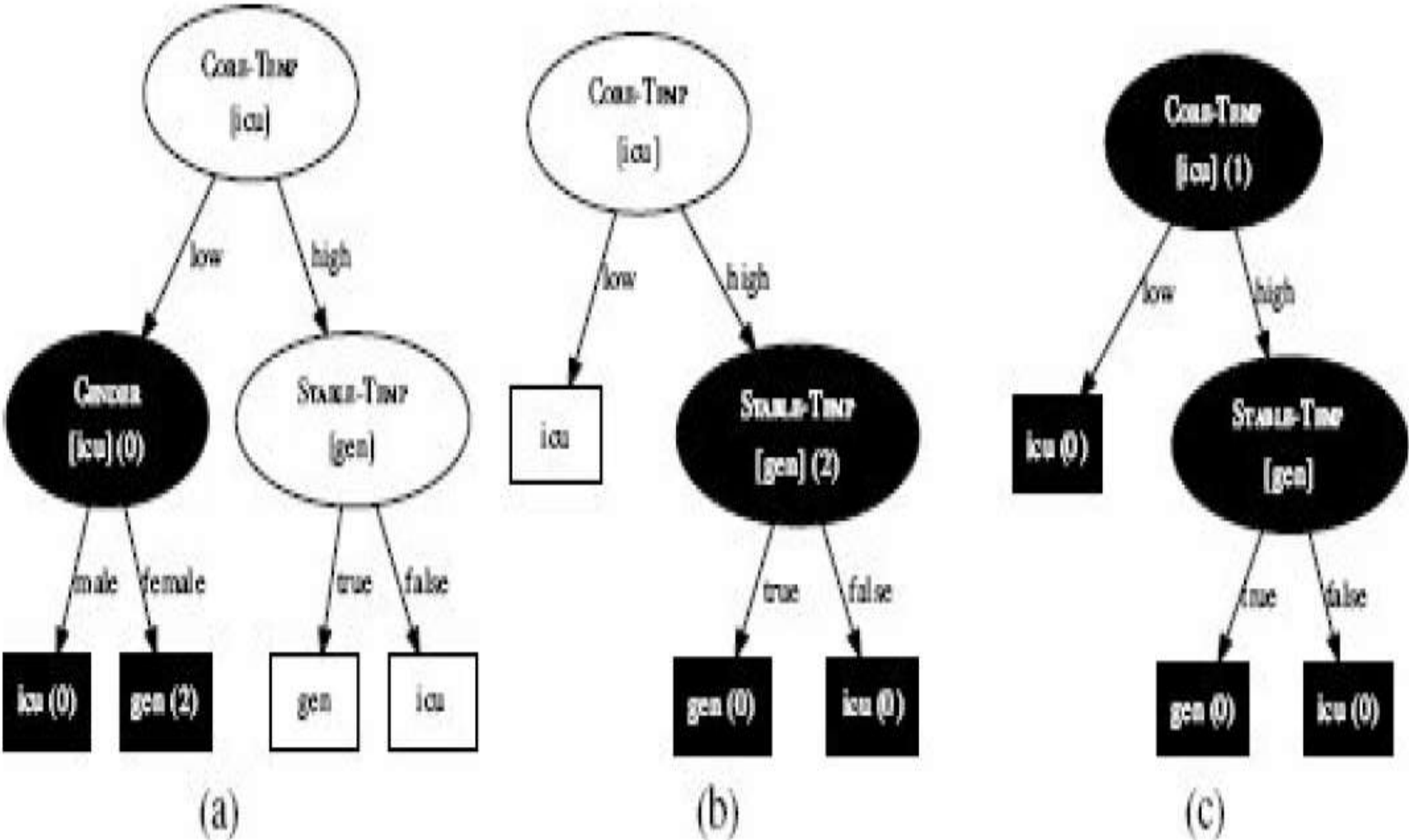
Figure: The decision tree for the post-operative patient routing task.

An example validation set for the post-operative patient routing task.

ID	CORE-TEMP	STABLE-TEMP	GENDER	DECISION
1	high	true	male	gen
2	low	true	female	icu
3	high	false	female	icu
4	high	false	male	icu
5	low	false	female	icu
6	low	true	male	icu

Table lists a validation dataset for this domain, and Figure below illustrate show this validation dataset is used to perform reduced error pruning. In Figure the pruning algorithm considers the subtree under the GENDER node for pruning. The path through the tree to make predictions for instances d2, d5, and d6 from the validation dataset leads to this subtree. The majority target level predicted at the root node of this subtree (the GENDER node) gives a correct prediction of *icu for each of the three instances*, so the error rate on the validation set for the root node of the subtree is 0. In contrast, the predictions made at the leaf nodes of this subtree are incorrect for d2 and d5 (because these patients are *female*, the prediction made is *gen* which does not match the validation dataset), so the error rate for the leaf nodes of this subtree is $0 + 2 = 2$. Because the error rate for the leaf nodes is higher than the error rate for the root node of the subtree, this subtree is pruned and replaced by a leaf node. The result of this pruning is visible on the left branch of the tree in Figure (b)

Figure:



The iterations of reduced error pruning for the decision tree in Figure using the validation set in Table . The subtree that is being considered for pruning in each iteration is highlighted in black. The prediction returned by each non-leaf node is listed in square brackets. The error rate for each node is given in round brackets.

In the second iteration of the algorithm, the subtree under the STABLE-TEMP node is considered for pruning (highlighted in Figure(b)). In this instance, the error rate for the root node of this subtree (the STABLE-TEMP node) is 2, whereas the error rate of the leaf nodes of the tree is $0 + 0 = 0$. As the error rate of the root node of the subtree is higher than the error rate of the leaf nodes, the tree is not pruned. Figure© illustrates the final iteration of the algorithm. In this iteration the subtree underneath the root node of the decision tree (the CORE-TEMP node) is considered for pruning (i.e., the full decision tree). In this iteration, the error rate of the root node (1) is greater than the error rate of the three leaf nodes, ($0 + 0 + 0 = 0$), so the tree is left unchanged.

Post-pruning using an error rate criteria is probably the most popular way to prune decision trees.²⁴ One of the advantages of pruning decision trees is that it keeps trees smaller, which in turn makes them easier to interpret. Another advantage is that pruning often increases the accuracy of the trees when there is noise in the training dataset. This is because pruning typically affects the lower parts of the decision tree, where noisy training data is most likely to cause overfitting. As such, pruning can be viewed as a noise dampening mechanism that removes nodes that have been created because of a small set of noisy instances.

➤ Model Ensembles

A prediction model that is composed of a set of models is called a model ensemble. The motivation behind using ensemble methods is the idea that a committee of experts working together on a problem are more likely to solve it successfully than a single expert working alone. As is always the case when a committee is working together, however, steps should be taken to guard against group think. In the context of ensemble models, this means that each model should make predictions independently of the other models in the ensemble. Given a large population of independent models, an ensemble can be very accurate even if the individual models in the ensemble perform only marginally better than random guessing.

- There are two defining characteristics of ensemble models:
 1. They build multiple different models from the same dataset by inducing each model using a modified version of the dataset.
 2. They make a prediction by aggregating the predictions of the different models in the ensemble. For categorical target features, this can be done using different types of voting mechanisms, and for continuous target features, this can be done using a measure of the central tendency of the different model predictions, such as the mean or the median.
- There are two standard approaches to creating ensembles: boosting and bagging

1. Boosting

When we use boosting, each new model added to an ensemble is biased to pay more attention to instances that previous models misclassified. This is done by incrementally adapting the dataset used to train the models. To do this we use a weighted dataset where each instance has an associated weight $w_i \geq 0$, initially set to $\frac{1}{n}$ where n is the number of instances in the dataset. These weights are used as a distribution over which the dataset is sampled to create a replicated training set, in which the number of times an instance is replicated is proportional to its weight.

Boosting works by iteratively creating models and adding them to the ensemble. The iteration stops when a predefined number of models have been added. During each iteration the algorithm does the following:

1. Induces a model using the weighted dataset and calculates the total error, ϵ , in the set of predictions made by the model for the instances in the training dataset. The ϵ value is calculated by summing the weights of the training instances for which the predictions made by the model are incorrect.
2. Increases the weights for the instances misclassified by the model using

$$w[i] \leftarrow w[i] \times \left(\frac{1}{2 \times \epsilon} \right)$$

and decreases the weights for the instances correctly classified by the model using

$$w[i] \leftarrow w[i] \times \left(\frac{1}{2 \times (1 - \epsilon)} \right)$$

- 3. Calculates a confidence factor, α , for the model such that α increases as ϵ decreases.
- A common way to calculate the confidence factor is

$$\alpha = \frac{1}{2} \times \log_e \left(\frac{1 - \epsilon}{\epsilon} \right)$$

Once the set of models has been created, the ensemble makes predictions using a weighted aggregate of the predictions made by the individual models. The weights used in this aggregation are the confidence factors associated with each model. For categorical target features, the ensemble returns the majority target level using a weighted vote, and for continuous target features, the ensemble returns the weighted mean.

2. Bagging

When we use bagging (or bootstrap aggregating), each model in the ensemble is trained on a random sample²⁸ of the dataset where, importantly, each random sample is the same size as the dataset and sampling with replacement is used. These random samples are known as bootstrap samples, and one model is induced from each bootstrap sample. The reason that we sample with replacement is that this will result in duplicates within each of the bootstrap samples, and consequently, every bootstrap sample will be missing some of the instances from the dataset. As a result, each bootstrap sample will be different, and this means that models trained on different bootstrap samples will also be different.

Decision tree induction algorithms are particularly well suited to use with bagging.

This is because decision trees are very sensitive to changes in the dataset: a small change in the dataset can result in a different feature being selected to split the dataset at the root, or high up in the tree, and this can have a ripple effect throughout the subtrees under this node. Frequently, when bagging is used with decision trees, the sampling process is extended so that each bootstrap sample only uses a randomly selected subset of the descriptive features in the dataset. This sampling of the feature set is known as subspace sampling. Subspace sampling further encourages the diversity of the trees within the ensemble and has the advantage of reducing the training time for each tree.

Figure below illustrates the process of creating a model ensemble using bagging and subspace sampling. The combination of bagging, subspace sampling, and decision trees is known as a random forest model. Once the individual models have been induced, the ensemble makes predictions by returning the majority vote or the median depending on the type of prediction required. For continuous target features, the median is preferred to the mean because the mean is more heavily affected by outliers

ID	F1	F2	F3	Target
1	-	-	-	-
2	-	-	-	-
3	-	-	-	-
4	-	-	-	-

**Bagging and
Subspace Sampling**

ID	F1	F3	Target
1	-	-	-
1	-	-	-
2	-	-	-
3	-	-	-

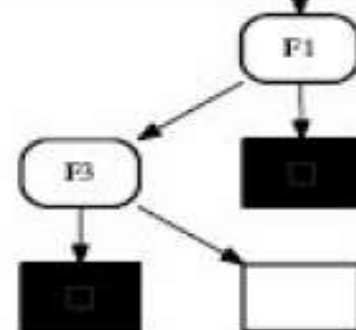
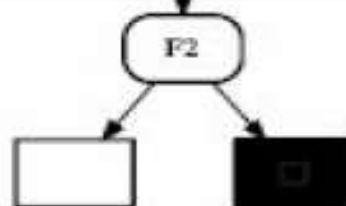
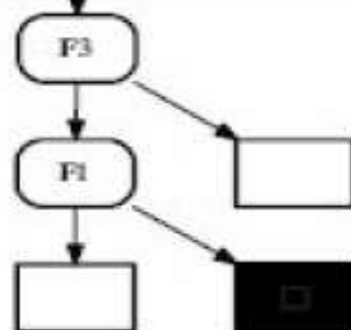
ID	F2	F3	Target
2	-	-	-
2	-	-	-
4	-	-	-
4	-	-	-

ID	F1	F3	Target
1	-	-	-
3	-	-	-
3	-	-	-
4	-	-	-

**Machine Learning
Algorithm**

**Machine Learning
Algorithm**

**Machine Learning
Algorithm**



MODEL ENSEMBLE